

AI-DRIVEN FULL STACK DEVELOPER

Zero to Hired

2026 Edition

Complete beginner to first developer job in 10–11 months

10–11

Months to hired

8

Phases

15+

Real projects

100%

Beginner friendly

codeWithShivah

Why learn to code when AI already writes code?



AI writes code. Yes.

GitHub Copilot, Claude, ChatGPT generate working code from one sentence. This is real. It is not going away. Ignoring it means falling behind.



But AI cannot think.

AI writes the HOW. You decide the WHAT and WHY. Architecture, product decisions, debugging hallucinations — still human work.



Your prompt quality = your output quality.

A developer who understands code writes 10x better prompts. You cannot review code you do not understand.



Market is shifting, not shrinking.

Companies want fewer average developers and more AI-native developers who ship 3x faster. That job is being created right now.



The new developer is an AI orchestrator.

You write 30% less code manually. Spend the rest reviewing, directing, and shipping. AI amplifies skill — it does not replace it.



The bottom line

Learn to code WITH AI, not instead of it. The developer who survives 2026 understands systems, thinks in solutions, and uses AI as a co-pilot — not a replacement for thinking.

Phase 1 **Computer Basics · Internet · Setup**

 Weeks 1–3 · ~3 hours/day

What to Study	Where to Learn (Free Resources)
• How computers work — CPU, RAM, Storage, OS	• CS50x — Harvard (edx.org) — FREE
• File system, folders, basic terminal / command prompt	• How Internet Works — Khan Academy — FREE
• How internet works — HTTP, DNS, browsers, servers	• VS Code Setup — Traversy Media (YouTube)
• What is a server vs client?	• Git Basics — FreeCodeCamp (YouTube)
• Install VS Code, Node.js, Git	• Touch typing — keybr.com — FREE
• Learn touch typing — aim for 30 WPM minimum	

Projects



My Dev Machine Setup Doc

Write a markdown README listing everything you installed and why. Push it to GitHub Week 3. First commit = start of your developer identity.



GitHub starts NOW — Week 3

Create GitHub account Week 1. Push your first commit Week 3. Even a README counts. Consistency from Day 1 builds a real contribution graph by Month 11.

Phase 2 **HTML · CSS · JavaScript**

 Weeks 4–9 · ~4 hours/day

What to Study	Where to Learn (Free Resources)
• HTML — semantic tags, forms, structure	• The Odin Project (theodinproject.com) — FREE
• CSS — flexbox, grid, responsive design, animations	• JavaScript.info — complete JS reference — FREE
• JS — variables, functions, DOM manipulation	• Kevin Powell — CSS on YouTube
• JS — arrays, objects, loops, conditions	• Akshay Saini — Namaste JavaScript (YouTube)
• JS — fetch API, async/await, Promises	• FreeCodeCamp — Responsive Web Design — FREE
• Browser DevTools — inspect, debug, network tab	

Projects



Personal Landing Page

Your name, photo, GitHub link. Deploy on GitHub Pages. First live URL.



Weather App

OpenWeather API + fetch + async/await. First real working web app.



LinkedIn — Start Week 7

Create profile after your first project is live. Headline: 'Learning Full Stack Dev | HTML CSS JS'. Post your weather app — screenshot + 3-line story. Connect with 5 developers per week.

Phase 3 **React.js · Portfolio · Git Workflow**

 Weeks 10–16 · ~4–5 hours/day

What to Study	Where to Learn (Free Resources)
• React — components, JSX, props, state	• react.dev — official docs (best resource) — FREE
• Hooks — useState, useEffect, useContext	• Codevolution — React playlist (YouTube)
• React Router — multi-page navigation	• tailwindcss.com — official docs — FREE
• Tailwind CSS — utility-first styling	• Learn Git Branching — interactive game — FREE
• Git branching — feature branches, PRs, merge	
• Deploy on Vercel / Netlify	

Projects



Task Manager App

CRUD with React state, local storage, Tailwind UI. Clean and functional.



Movie Search App

OMDB API + React Router + favorites. Shows real frontend depth.



Build Portfolio — Week 14

Build in React. Sections: About, Skills, Projects, Contact. Show 3 projects with live links + GitHub. Deploy on Vercel. Update LinkedIn to 'React Developer | Building Full Stack'.

Phase 4 **Node.js · Express · MongoDB**

 Weeks 17–24 · ~5 hours/day

What to Study	Where to Learn (Free Resources)
• Node.js — runtime, modules, npm ecosystem	• Codevolution — Node.js full course (YouTube)
• Express.js — routes, middleware, REST API design	• expressjs.com — official docs — FREE
• MongoDB — documents, collections, CRUD	• MongoDB University (learn.mongodb.com) — FREE
• Mongoose — schemas, models, validation	• Dave Gray — REST API Design (YouTube)
• Authentication — JWT, bcrypt, sessions	• Traversy Media — JWT Auth (YouTube)
• Error handling, status codes, API conventions	

Projects



E-Commerce Backend

User auth, product CRUD, cart, orders — full REST API with JWT. Deploy on Render.



Blog API

Users, posts, comments, likes. Mongoose relationships. Document with Postman.



GitHub — Week 20 onwards

Commit every day. Write proper README for every project. Use feature branches. By Week 24: 5+ repos, 60+ commits, green graph.

Phase 5 **Full Stack MERN + AI Features**

 Weeks 25–32 · ~5–6 hours/day

What to Study	Where to Learn (Free Resources)
• Connect React frontend to Express backend	• redux-toolkit.js.org — official docs — FREE
• State management — Redux Toolkit or Zustand	• Socket.io docs + Traversy Media (YouTube)
• File uploads — Multer + Cloudinary	• OpenAI API Docs — FREE
• Real-time — Socket.io basics	• TechWorld with Nana — Docker (YouTube)
• OpenAI / Claude API — add AI to your app	
• Docker basics — containerize your app	

Projects



AI-Powered Chat App

Full MERN + Socket.io real-time + Claude/OpenAI API chatbot + file upload + auth. Deploy on Render + Vercel. This is your headline project.



Full Stack E-Commerce (Complete)

Phase 4 backend + React frontend + Razorpay payment + admin panel + AI product recommendation. Shows end-to-end thinking.



LinkedIn Week 30 — Full Stack Developer | Open to Work

Post a demo video of your AI Chat App. Tag the stack. This is the post recruiters respond to.

Phase 6 **DSA + Technical Interview Prep**

 Weeks 33–36 · 4 weeks dedicated · Month 9

Week	Focus + Task
Week 33	Arrays + Strings + Hashmaps Solve 15 easy LeetCode problems. Focus on patterns — sliding window, two pointers. Not memorizing, understanding.
Week 34	Recursion + Sorting + Linked Lists 10 more problems. Understand time complexity $O(n)$, $O(n^2)$, $O(\log n)$. Know why it matters for interviews.
Week 35	Stack, Queue, Binary Search, Trees (basics) 10 problems. Focus on patterns, not grinding. 50 easy problems total is enough for most product companies at fresher level.
Week 36	Mock Interviews + Review Weak Areas Pramp.com or interviewing.io — 3 mock sessions. Time yourself. Explain your thinking out loud.

Technical Concepts to Revise

- JavaScript — closures, hoisting, event loop, this keyword
- React — virtual DOM, reconciliation, re-renders, hooks rules
- Node.js — event-driven, non-blocking I/O, middleware chain
- REST API — methods, status codes, CRUD conventions
- Database — indexing basics, schema design, Mongoose relationships
- System design basics — what happens when you type google.com
- Resume — one page, Jake's Resume template, 3 projects with live links
- Apply on: LinkedIn Jobs, Internshala, Naukri, AngelList, Wellfound

Phase 7 **English + Communication for Developers**

 30 minutes every single day · Start from Month 3, intensify Month 10

The honest reality

Your English does not need to be perfect. It needs to be clear and confident. Thousands of Indian developers work at Google, Amazon, Microsoft with Indian-accented English. The problem is never the accent — it is hesitation, low vocabulary, and no practice. This phase fixes that.

Daily 30-Minute Tasks (Every Day from Month 3)

T1 Speak, don't type — 10 min

Explain what you coded today in English. Out loud. Record yourself on your phone. 'Today I built a login form using React. I used useState to track the input values.' No audience needed. Habit builds fluency.

T2 Read 1 dev blog post — 10 min

dev.to, medium.com, css-tricks.com — read in English. Builds vocabulary naturally. You will start thinking in dev English without realising it.

T3 Write 3 sentences on LinkedIn — 5 min

Post or comment in English — even broken English is fine at the start. Consistency beats perfection. Every week your writing improves.

T4 Watch 1 English YouTube video — 5 min

With English subtitles on. Fireship, Theo, Kevin Powell, Traversy — all informal English used by real developers. Shadow their sentence patterns.

Interview-Specific English Practice

Prepare your 'Tell me about yourself' in English

Write it first in Hindi. Translate line by line. Practice saying it 20 times until it flows naturally. This one answer decides the tone of every interview.

Explain your projects out loud — in English

'I built a full stack e-commerce app using React, Node, and MongoDB. The main challenge was handling JWT token expiry. I solved it by implementing refresh tokens.' Practice this for every project.

Mock interview in English — Month 10

Pramp.com forces English. Or find a friend and practice. Record it. Watch back once — painful but the fastest way to improve.

Think-aloud practice for coding problems

'I'm thinking of using a hashmap here because the lookup time is O(1)...' Interviewers value process over answer. Practice narrating your thinking in English.

Best Free Tools for English Improvement

- ELSA Speak app — AI pronunciation coach. 15 min/day. (Free tier available)
- BBC Learning English (YouTube) — clear, professional English
- Anki flashcards — build a dev vocabulary deck of 200 words — FREE
- ChatGPT / Claude — practice interview conversations, get corrections — FREE
- Grammarly — paste your LinkedIn posts before publishing — FREE

Phase 8 Applications · Offers · First Job

 Weeks 41–44 · Month 11 · Full focus mode

Where to Apply	Resume — One Page, No Fluff
• LinkedIn Jobs — Easy Apply	• Skills: React, Node, Express, MongoDB, Git, REST, Docker
• Internshala — fresher roles and internships	• 3 projects with live links + tech stack listed
• Naukri.com — mark 'Actively Looking'	• No 'team player', 'passionate' — show, don't tell
• AngelList / Wellfound — startups move faster	• GitHub link + Portfolio link — mandatory
• Direct outreach — DM HR on LinkedIn after applying	• Use Jake's Resume template — ATS-friendly, open source
• Referrals — ask your network from Phase 5	• One page only. Recruiters spend 30 seconds on it.

Interview Rounds You'll Face

- Round 1: Online DSA test — 2–3 easy/medium problems
- Round 2: Technical interview — JS/React/Node concepts + 1 DSA
- Round 3: Project walkthrough — explain your code and decisions
- Round 4: HR round — Tell me about yourself, salary, notice period

Salary Expectations (Fresher, India 2026)

- Service companies (TCS, Infosys, Wipro) — ₹3.5–5 LPA
- Product startups — ₹6–12 LPA
- Mid-size product companies — ₹8–15 LPA
- Remote international roles — ₹15–30 LPA
- With AI portfolio projects — you negotiate from strength

The golden rule

Your GitHub, portfolio, and LinkedIn do 70% of the selling before you enter any interview room. Keep all three updated until the day you sign your offer letter.

11-Month Roadmap Timeline

Month 1	Phase 1 — Computer Basics + Setup  GitHub account created  First commit pushed
Month 2	Phase 2 — HTML + CSS + JavaScript  Weather App on GitHub  LinkedIn created  First project post
Month 3–4	Phase 3 — React + Portfolio  Portfolio live  React Developer on LinkedIn  3 repos pinned
Month 5–6	Phase 4 — Node.js + Express + MongoDB  Backend repos with README  'Built my first REST API' post
Month 7–8	Phase 5 — Full Stack MERN + AI Features  AI Chat App — flagship project  Full Stack Dev Open to Work  Portfolio updated
Month 9	Phase 6 — DSA + Technical Interview Prep  50 LeetCode problems  JS/React/Node deep dive  3 mock interviews
Month 10	Phase 7 — English + Communication  Daily 30-min speaking practice  Interview scripts prepared  Mock interviews in English
Month 11	Phase 8 — Applications + First Job  5–10 applications/day  100+ commits, green graph  First offer received

GitHub · LinkedIn · Portfolio

 GitHub — Start Week 1	 LinkedIn — Start Week 7	 Portfolio — Build Month 3
• Create account Week 1. Push first file Week 3. • Commit something every day — even docs count. • Write a proper README for every project. • Pin your 4–6 best repos on profile page. • Professional commit messages always. • Goal by Month 11: 100+ commits, green graph, 8+ repos.	• Don't create until you have something to show. • Professional photo. Strong headline. Clear summary. • Post every project as a milestone — screenshot + story. • Engage with dev content daily — comment with substance. • Connect 5–10 devs per week. Give before you ask. • Turn on 'Open to Work' in Month 8.	• Don't build Month 1 — nothing to put in it yet. • Build in React. Keep it fast and minimal. • Maximum 3 projects — your best only. Always. • Each project: live demo + GitHub + 2-line description. • Contact form or email link — make it easy. • Deploy on Vercel. Custom domain if possible.

How to Build Your Network

01

Start Twitter/X + LinkedIn — Month 2

Follow developers 1–2 years ahead of you. Comment with substance. 'I tried this and got stuck on X — what helped you?' Show up consistently. People notice before they follow.

02

Build in public — share the struggle, not just wins

'I spent 3 hours debugging this CORS error and here's what finally fixed it' gets more engagement than 'I built a project.' The struggle is relatable. The solution is valuable.

03

Join communities — Discord, Reddit — Month 3

The Odin Project Discord. 100Devs. Local tech meetups. Answer what you know. Ask what you don't. Help one person a day — it compounds into reputation.

04

Cold outreach — Month 7 onwards

DM 3–5 developers weekly. Not 'please refer me.' Instead: 'I saw your post on React performance. I'm a MERN dev open to work. Would love any advice.' Short. Specific. Humble.

05

Contribute to open source — Month 6 onwards

Fix a small bug. Improve docs. Raise a PR. Even one merged PR puts you in a different category. It shows you can work in a codebase you didn't build yourself.

How to Use AI While Learning

✓ USE AI FOR	✗ DON'T USE AI FOR
• Explaining concepts — paste code, ask 'explain line by line like I'm 15'	• Write your projects for you — you learn nothing, you will fail interviews
• Debugging help — paste your error + code, ask 'what's wrong and why'	• Skip understanding — if you can't explain your own code, the interviewer knows
• Code review — after finishing a project, ask what a senior dev would change	• Paste 'build me a full stack todo app' and copy the output without reading it
• English practice — paste your LinkedIn post, ask 'is this clear and professional'	• Use AI to avoid thinking — AI is the gym partner, not the person doing your reps
• Interview prep — practice 'Tell me about yourself' and get corrections	

⚡ The bottom line on AI

The developer who gets hired in 2026 uses AI to move faster — not to avoid learning. Your understanding is the product. AI is just the tool.



Now go build something.

Every developer you look up to started exactly where you are right now — with a blank screen, zero clue, and a stubborn decision to not give up. The roadmap is clear. The resources are free. The only variable is consistency.

30 min

English. Every day.

1 commit

GitHub. Every day.

1 project

Each phase. Always.

codeWithShivah

Full Stack Developer Roadmap · 2026 Edition · 100% Free